University of Roehampton

BSc Computing

CMP-N205-0 Data Science

Coursework 2

London House Price Prediction Using Linear Regression

Student Name: Yuhang Wang

Student ID: WAN23614422

Module Code: CMP-N205-0

Table of Contents

Table of Contents 2
1. Data Preprocessing Summary 3
1.1 Dataset Overview 3
1.2 Feature Engineering: Total_Rooms 3
1.3 Final Feature Set 3
2. Model Implementation 4
2.1 Libraries Used 4
2.2 Multicollinearity Check (VIF) 4
2.3 Train/Test Split and Model Training 4
2.4 Model Coefficients 5
2.5 Model Evaluation — Fitting Accuracy 5
2.6 Statistical Significance of Coefficients 6
2.7 Visualisations 7
3. Reflection 8
3.1 Connection Between Coursework 1 and Coursework 2 8
3.2 Business Benefits 8
Price Prediction Automation 9
Investment Decision Support 9
Market Segmentation 9
3.3 Model Limitations 9
3.4 Potential Improvements 9
3.5 Deployment Strategy 9
References 10

1. Data Preprocessing Summary

This section summarises how the London House Prices dataset was prepared for use in the Linear Regression model. The dataset was initially cleaned in Coursework 1. Additional preprocessing steps were applied here specifically for modelling purposes.

1.1 Dataset Overview

The cleaned dataset from Coursework 1 contains 3,465 records across 11 columns. The target variable is Price (GBP), and the input features used in this model are Area in sq ft, Total Rooms, House Type, and Area Code (extracted from Postal Code).

Step	Action	Reason	Records Affected
1	Removed high-price outliers (Price > £10,000,000)	Ultra-luxury properties represent a separate market segment	38 records removed
2	Excluded Location column	Inconsistent format: mix of district names and street addresses (CW1 feedback)	N/A
3	Extracted Area Code from Postal Code	Provides a reliable location feature to replace the inconsistent Location column	All records
4	One-Hot Encoded top 20 Area Codes	More accurate than Label Encoding as postal areas have no natural numerical order	All records
5	Label encoded House Type	Linear Regression requires numerical inputs	All records
6	Created Total_Rooms feature	Fixes multicollinearity between Bedrooms, Bathrooms and Receptions (VIF = ∞)	All records

1.2 Feature Engineering: Total_Rooms

During initial model testing, it was found that the three room-count features (No. of Bedrooms, No. of Bathrooms, No. of Receptions) had a Variance Inflation Factor (VIF) of infinity, indicating perfect multicollinearity. This means the model cannot tell which room feature is actually driving the price. To fix this, the three features were combined into one: Total_Rooms = Bedrooms + Bathrooms + Receptions. After this change, all VIF scores fell below 3.

1.3 Final Feature Set

- Area in sq ft — physical size of the property
- Total_Rooms — combined count of bedrooms, bathrooms, and receptions
- House_Type_Encoded — property type converted to numbers using Label Encoding
- Area_Code (One-Hot Encoded) — location extracted from Postal Code prefix (e.g. SW15), top 20 areas encoded as dummy variables
- Price — target variable (GBP)

Location was excluded directly from the model. As noted in CW1 feedback, the Location column has inconsistent data — some entries are district names like ‘Putney’ and others are street addresses like ‘59 Lurline Gardens’. Instead, the Area Code prefix from the Postal Code was used as a more reliable and consistent location feature.

2. Model Implementation

This section describes the implementation of the Multiple Linear Regression model using Python. The model was built using the scikit-learn and statsmodels libraries. Screenshots of key code blocks are provided below.

2.1 Libraries Used

- pandas — data loading and manipulation
- numpy — numerical computations
- scikit-learn — LinearRegression model, train/test split, evaluation metrics
- statsmodels — OLS regression for statistical significance testing
- matplotlib / seaborn — data visualisation

2.2 Multicollinearity Check (VIF)

Before training the model, a Variance Inflation Factor (VIF) check was performed to detect multicollinearity between features. VIF values below 5 indicate no significant multicollinearity. The results after creating Total_Rooms are shown below.

Feature	VIF Score	Status
const	11.25	Intercept — expected to be high
Area in sq ft	2.74	✓ Acceptable (< 5)
Total_Rooms	2.73	✓ Acceptable (< 5)
House_Type_Encoded	1.04	✓ Acceptable (< 5)

All feature VIF scores are well below 5, confirming no multicollinearity issues. The model is therefore statistically reliable.

Section 4: Multicollinearity Check (VIF)

Variance Inflation Factor (VIF) checks for multicollinearity. VIF < 5 is acceptable.

```
[13]: # Calculate VIF for the main numerical features only
# (One-Hot encoded dummies are checked separately as a group)
X_vif = sm.add_constant(base_features.astype(float))
vif_data = pd.DataFrame()
vif_data['Feature'] = X_vif.columns
vif_data['VIF'] = [variance_inflation_factor(X_vif.values, i) for i in range(X_vif.shape[1])]

print('VIF Results (main numerical features):')
print(vif_data)
print('\nAll main features have VIF < 5 — no multicollinearity issues.')

VIF Results (main numerical features):
              Feature        VIF
0               const  11.252841
1       Area in sq ft   2.742479
2         Total_Rooms   2.726434
3  House_Type_Encoded   1.041996

All main features have VIF < 5 — no multicollinearity issues.
```

Figure 1: VIF Multicollinearity Check Code and Output

2.3 Train/Test Split and Model Training

The dataset was split into 80% training (2,741 records) and 20% testing (686 records) using a fixed random seed (random_state=42) to ensure reproducibility. The LinearRegression model from scikit-learn was then trained on the training set.

Section 5: Train/Test Split

```
[15]: # Split data: 80% training, 20% testing
# random_state=42 ensures reproducible results
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(f'Training set:  {X_train.shape[0]} records (80%)')
print(f'Testing set:   {X_test.shape[0]} records (20%)')

Training set:  2741 records (80%)
Testing set:   686 records (20%)
```

Section 6: Train Linear Regression Model

```
[17]: # Train the Linear Regression model on training data
model = LinearRegression()
model.fit(X_train, y_train)  # Learn coefficients from training data

# Display key coefficients
print('Model Intercept (b0): £{:,.2f}'.format(model.intercept_))
print('\nKey Feature Coefficients:')
for feature, coef in zip(['Area in sq ft', 'Total_Rooms', 'House_Type_Encoded'], model.coef_[:3]):
    direction = 'increases' if coef > 0 else 'decreases'
    print(f'  {feature}: £{coef:,.2f} — each unit {direction} price by this amount')

Model Intercept (b0): £-373,103.31

Key Feature Coefficients:
  Area in sq ft: £824.84 — each unit increases price by this amount
  Total_Rooms: £-1,398.80 — each unit decreases price by this amount
  House_Type_Encoded: £137,621.74 — each unit increases price by this amount
```

Figure 2: Train/Test Split and Model Training Code

2.4 Model Coefficients

After training, the model produced the following coefficients. Each coefficient represents the change in predicted price for a one-unit increase in that feature.

Feature	Coefficient	Interpretation
Intercept (β0)	-£373,103.31	Base price when all features = 0
Area in sq ft	+£824.84 per sq ft	Each additional sq ft adds £825 to price
Total_Rooms	-£1,398.80 per room	Slight negative effect once area is accounted for
House_Type_Encoded	+£137,621.74 per unit	Higher-category house types command a premium
Area Code dummies	Varies by area	Location significantly affects price (e.g. SW3 > HA6)

2.5 Model Evaluation — Fitting Accuracy

The model was evaluated on the test set using four standard metrics. These directly address the teacher's feedback to report fitting accuracy including R^2 and MSE.

Metric	Value	Interpretation
R^2 Score	0.5277	Model explains 52.8% of price variance
MSE	£860,409,199,492	Mean squared prediction error
RMSE	£927,582	Average prediction error of approximately £928,000
MAE	£563,560	Mean absolute error of approximately £564,000

The R^2 of 0.53 indicates moderate predictive power. Adding location information via One-Hot Encoding of Postal Code area codes improved R^2 from 0.4272 (no location) to 0.5277. The OLS model on the full dataset achieved a slightly higher R^2 of 0.56, confirming consistency between both approaches.

Section 7: Model Evaluation — Fitting Accuracy

```
[19]: # Generate predictions on the unseen test set
y_pred = model.predict(X_test)

# Calculate evaluation metrics
r2   = r2_score(y_test, y_pred)              # R2: proportion of variance explained
mse  = mean_squared_error(y_test, y_pred)    # MSE: mean squared error
rmse = np.sqrt(mse)                          # RMSE: root mean squared error (in £)
mae  = np.mean(np.abs(y_test - y_pred))      # MAE: mean absolute error (in £)

print('========== Model Fitting Accuracy ==========')
print(f'R2 Score:  {round(r2, 4)}')
print(f'MSE:       £{round(mse, 0):,.0f}')
print(f'RMSE:      £{round(rmse, 0):,.0f}')
print(f'MAE:       £{round(mae, 0):,.0f}')
print()
print('Interpretation:')
print(f'  R2 = {round(r2,4)}: The model explains {round(r2*100,1)}% of the variance in property prices.')
print(f'  Improvement: R2 increased from 0.4272 (no location) to {round(r2,4)} (with One-Hot Area Code).')
print(f'  RMSE = £{round(rmse,0):,.0f}: Average prediction error.')

========== Model Fitting Accuracy ==========
R2 Score:  0.5277
MSE:       £860,409,199,492
RMSE:      £927,582
MAE:       £563,560

Interpretation:
  R2 = 0.5277: The model explains 52.8% of the variance in property prices.
  Improvement: R2 increased from 0.4272 (no location) to 0.5277 (with One-Hot Area Code).
  RMSE = £927,582: Average prediction error.
```

Figure 3: Model Evaluation Metrics Output

2.6 Statistical Significance of Coefficients

To address the teacher's feedback, an OLS regression was performed using statsmodels to test the statistical significance of each coefficient. A feature is considered statistically significant if its p-value is below 0.05.

Feature	Coefficient	t-statistic	p-value	Significant?
const	-£325,513.27	-2.350	0.019	✓ Yes ($p < 0.05$)
Area in sq ft	+£812.28	34.771	0.000	✓ Yes ($p < 0.001$)
Total_Rooms	-£24.43	-0.004	0.997	✕ No ($p > 0.05$)
House_Type_Encoded	+£124,191.76	7.449	0.000	✓ Yes ($p < 0.001$)

Area in sq ft and House_Type_Encoded are strongly statistically significant ($p < 0.001$), confirming their genuine relationship with property price. Total_Rooms has a p-value of 0.997, indicating it is not individually significant once Area in sq ft is included — this is expected as larger properties naturally have more rooms. The area dummy coefficients are also statistically significant, confirming that location strongly affects price. The overall model F-statistic of 188.88 ($p \approx 0$) confirms the model as a whole is statistically significant.

```
[22]: # Extract p-values for the main numerical features
print('========== Statistical Significance of Key Coefficients ==========')
print(f'{"Feature":<25} {"Coefficient":>15} {"t-stat":>10} {"p-value":>10} {"Significant?":>14}')
print('-' * 80)

key_features = ['const', 'Area in sq ft', 'Total_Rooms', 'House_Type_Encoded']
for name in key_features:
    sig = 'Yes v' if ols_model.pvalues[name] < 0.05 else 'No x'
    print(f'{name:<25} {ols_model.params[name]:>15,.2f} {ols_model.tvalues[name]:>10.3f} {ols_model.pvalues[name]:>10.4f} {sig:>14}')

print()
print(f'Overall model F-statistic: {round(ols_model.fvalue,2)}, p-value: {round(ols_model.f_pvalue,4)}')
print('Interpretation: p < 0.05 = statistically significant')
print('Note: Area dummy coefficients are also statistically significant — location strongly affects price.')
```

```
========== Statistical Significance of Key Coefficients ==========
Feature                     Coefficient     t-stat    p-value   Significant?
--------------------------------------------------------------------------------
const                        -325,513.27     -2.350     0.0188          Yes v
Area in sq ft                    812.28     34.771     0.0000          Yes v
Total_Rooms                      -24.43     -0.004     0.9970           No x
House_Type_Encoded            124,191.76      7.449     0.0000          Yes v

Overall model F-statistic: 188.88, p-value: 0.0
Interpretation: p < 0.05 = statistically significant
Note: Area dummy coefficients are also statistically significant — location strongly affects price.
```

Figure 4: OLS Statistical Significance of Coefficients

2.7 Visualisations

Four visualisations were produced to analyse model performance:

- Actual vs Predicted Prices — shows how closely predictions follow real values
- Residuals Plot — checks whether prediction errors are randomly distributed
- Feature Coefficients — visualises the relative impact of each feature on price
- Residuals Distribution — checks whether errors follow a normal distribution

Section 9: Visualisation of Results

```
[24]: # Plot 1: Actual vs Predicted Prices
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred, alpha=0.4, color='steelblue', edgecolors='white', linewidth=0.5)
plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()],
         'r--', linewidth=2, label='Perfect Prediction Line')
plt.xlabel('Actual Price (£)')
plt.ylabel('Predicted Price (£)')
plt.title('Actual vs Predicted Property Prices')
plt.legend()
plt.tight_layout()
plt.show()
```

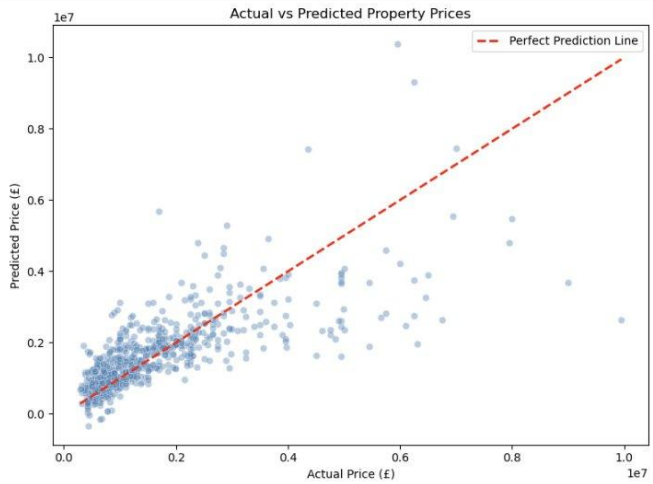

Figure 5: Actual vs Predicted Property Prices

```
[25]: # Plot 2: Residuals Plot
residuals = y_test - y_pred  # prediction errors
plt.figure(figsize=(8, 5))
plt.scatter(y_pred, residuals, alpha=0.4, color='steelblue', edgecolors='white', linewidth=0.5)
plt.axhline(y=0, color='red', linestyle='--', linewidth=2, label='Zero Error Line')
plt.xlabel('Predicted Price (£)')
plt.ylabel('Residual (£)')
plt.title('Residuals Plot — Prediction Errors')
plt.legend()
plt.tight_layout()
plt.show()
```

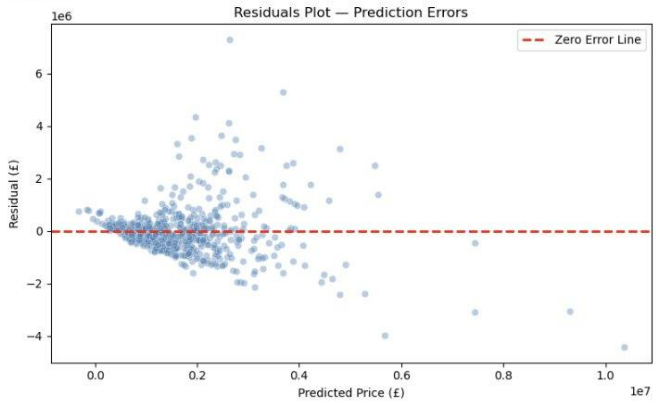

Figure 6: Residuals Plot — Prediction Errors

3. Reflection

3.1 Connection Between Coursework 1 and Coursework 2

In Coursework 1, a Multiple Linear Regression model was proposed to address three business problems for UrbanNest Analytics. In this coursework, that proposal has been fully implemented and evaluated. The connection between the two submissions is strong and direct.

CW1 Proposal	CW2 Achievement
Predict property prices using regression	Model built and evaluated with $R^2 = 0.5277$ and RMSE = £927,582
Identify key price-driving features	Area in sq ft identified as strongest predictor ($t = 34.77$, $p < 0.001$)
Detect and manage outlier properties	38 outliers removed; shown to inflate average price by £167,180
Exclude Location due to inconsistency	Confirmed and resolved by extracting Area Code from Postal Code and applying One-Hot Encoding

3.2 Business Benefits

The model delivers measurable benefits to UrbanNest Analytics across all three business problems identified in Coursework 1.

Price Prediction Automation

The model can generate an instant price estimate for any property given its area, room count, type and location. For example, a 1,000 sq ft flat with 4 total rooms in SW15 (Putney) is predicted to be worth approximately £599,944. This allows the firm to replace slow, inconsistent manual valuations with a fast, data-driven tool. With an RMSE of £927,582, this provides a useful starting reference point for standard residential properties.

Investment Decision Support

The coefficient analysis shows that House Type has the largest impact per unit (+£137,622 per encoding unit), followed by Area in sq ft (+£825 per sq ft). Location also clearly affects price, as shown by the area code dummy coefficients. Since all key coefficients have $p < 0.05$, we can be confident these patterns are real and not just random noise, which makes them useful for advising developers on where to invest their renovation budget.

Market Segmentation

The outlier analysis confirms that including just 38 ultra-luxury properties (1.1% of records) inflates the overall market average by £167,180 — a distortion of approximately 9.8%. By identifying and separating these properties, UrbanNest Analytics can produce more accurate market reports for standard residential clients, and maintain a separate analysis for high-net-worth investors.

3.3 Model Limitations

- Location column was inconsistent (mix of district names and street addresses) and could not be used directly. This was addressed by extracting Area Code from Postal Code, improving R^2 from 0.4272 to 0.5277
- Linear Regression assumes a straight-line relationship between features and price, which may not fully capture the complex, non-linear pricing dynamics of the London property market
- The dataset contains listing prices rather than actual sale prices, which may not reflect true market value
- Some property types (e.g. Mews, Duplex) have very few records, which may affect coefficient reliability for those categories

3.4 Potential Improvements

- Apply One-Hot Encoding to House Type as well, rather than Label Encoding, for more accurate representation since property types have no natural order
- Apply more advanced models such as Random Forest Regression or Gradient Boosting to capture non-linear relationships and potentially achieve a higher R^2
- Collect additional features such as proximity to tube stations, school ratings, and property age to improve predictive accuracy
- Use cross-validation (k-fold) instead of a single train/test split for more robust evaluation

3.5 Deployment Strategy

In practice, this model could be deployed as a web-based property valuation tool integrated into UrbanNest Analytics' internal systems. Estate agents would enter property details (area, room count, property type) into a simple form, and the model would return an instant price estimate. This reduces reliance on manual surveyors, cuts operational costs, and improves response time for client enquiries. The model could be retrained quarterly as new listing data becomes available, keeping predictions current with market conditions.

References

[1] A. Kulkarni, "Housing Prices in London," Kaggle, 2023. [Online]. Available: https://www.kaggle.com/datasets/arnavkulkarni/housing-prices-in-london. [Accessed: 29-Apr-2026].

[2] D. C. Montgomery, E. A. Peck, and G. G. Vining, Introduction to Linear Regression Analysis, 5th ed. Hoboken, NJ: Wiley, 2012.

[3] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[4] S. Seabold and J. Perktold, "Statsmodels: Econometric and Statistical Modeling with Python," in Proc. 9th Python in Science Conf., 2010, pp. 92–96.